

Congestion Control

Congestion control is handled as follows:

Slow start: Upon receiving an ACK, guaranteeing that the ACK number is correct (ie, greater than the previously received ACK), and that the current cwnd is under ssthresh, then, the cwnd will be increased by the number of A - the most recently received ACK.

Congestion avoidance: The logic works similarly to slow start, but congestion avoidance occurs if the cwnd is NOT under ssthresh. In this case, the cwnd will increase by $(\# \text{ of acks received} / \text{cwnd})$, to be linear.

Fast retransmit: If the client receives an ACK which is duplicated, the client will increment a duplicate ACK counter (which is reset to 0 once it reaches a different ACK). Then, this duplicate ACK counter is checked, if it is 3, fast retransmit occurs, and the client sends the missing packet. Additionally, ssthresh is set to $\frac{1}{2}$ cwnd (although just as a sanity check, the client's code uses a max with 1 in case cwnd is 1), and cwnd is set to the new ssthresh + 3.

Timeouts and Retransmissions

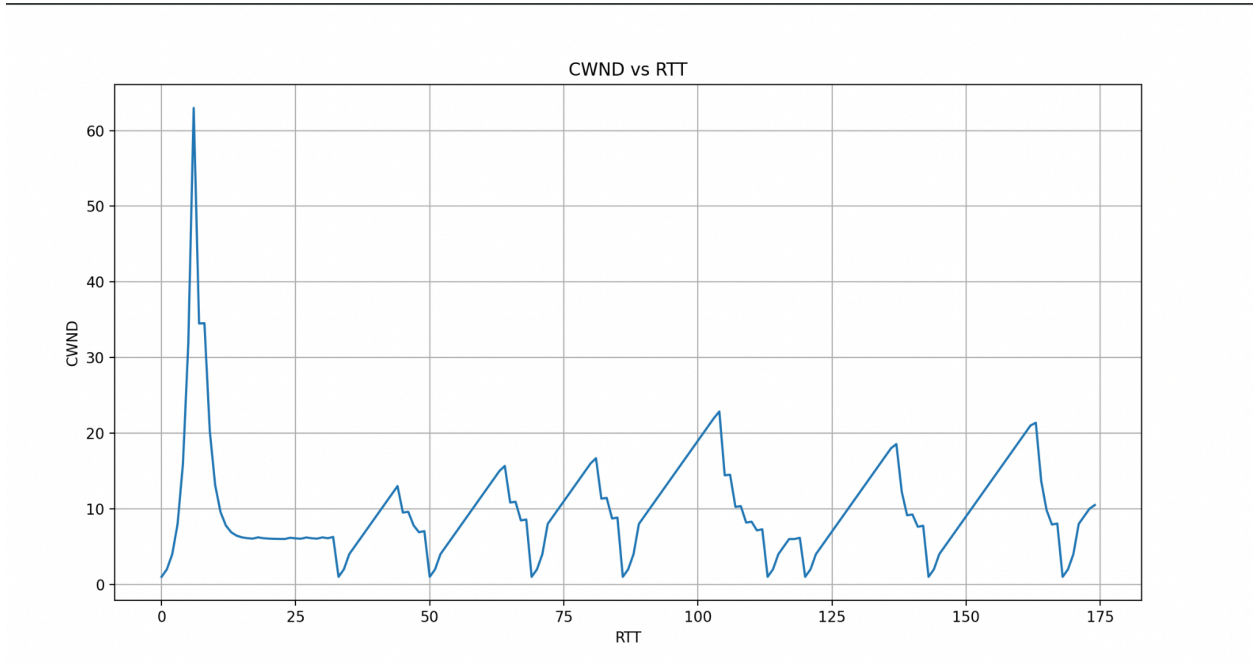
Timeouts and retransmissions due to timeouts were handled in a specific thread. The client would store its most recently received ACK number and at the time that ACK came in. This timeout handler would periodically check if 500 ms had passed from the previous ACK (And in the receiving ACK thread, upon receiving an ACK, these variables would be updated). If 500 ms had passed, then the client would retransmit the packet, setting the ssthresh to be the original cwnd/2, and resetting the cwnd to 1.

(However, retransmissions due to fast retransmit were in the client's thread for the purposes of receiving ACKs, because ACKs coming in were the signal that fast retransmit should occur.)

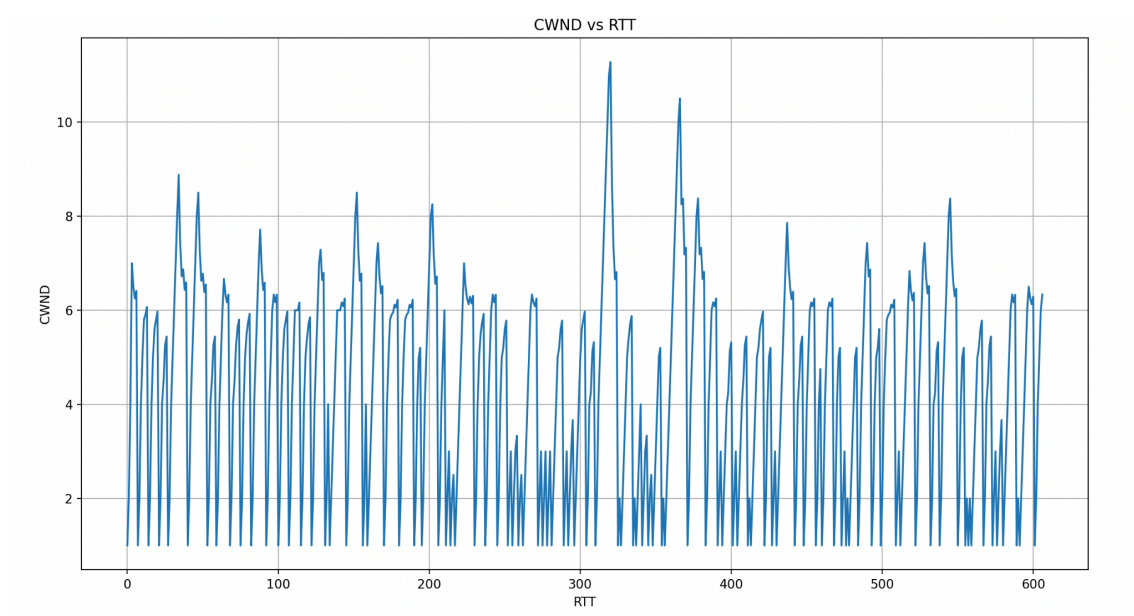
Graphs

CWND vs RTT

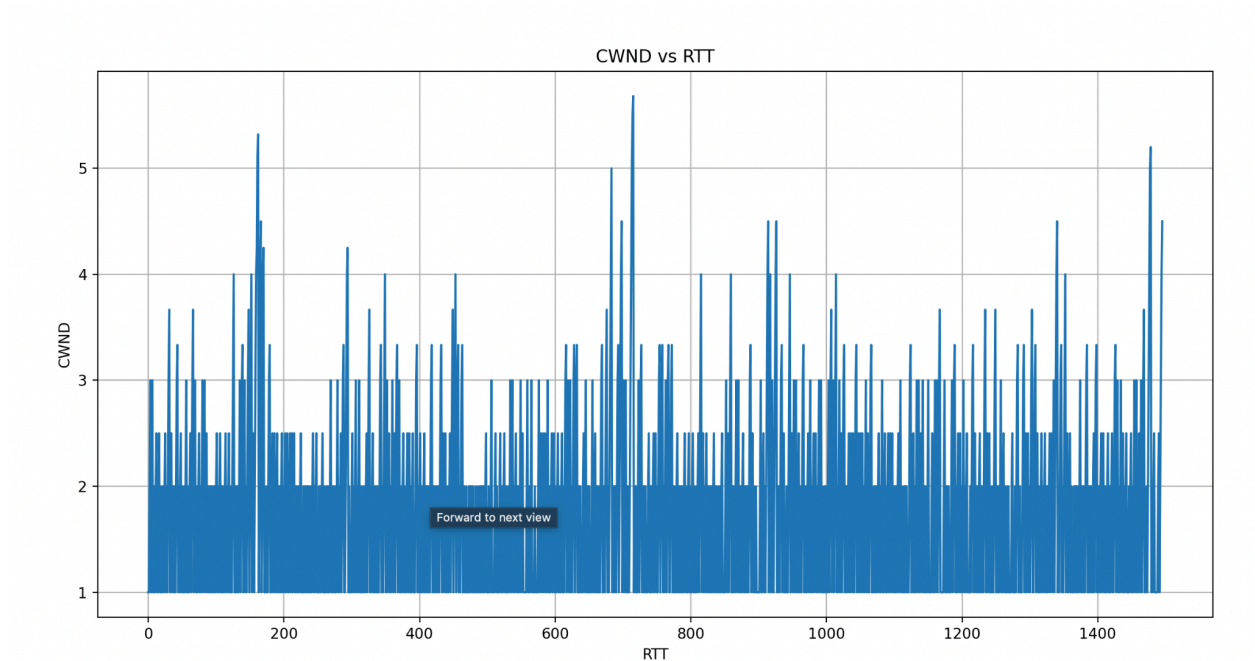
1%



10%

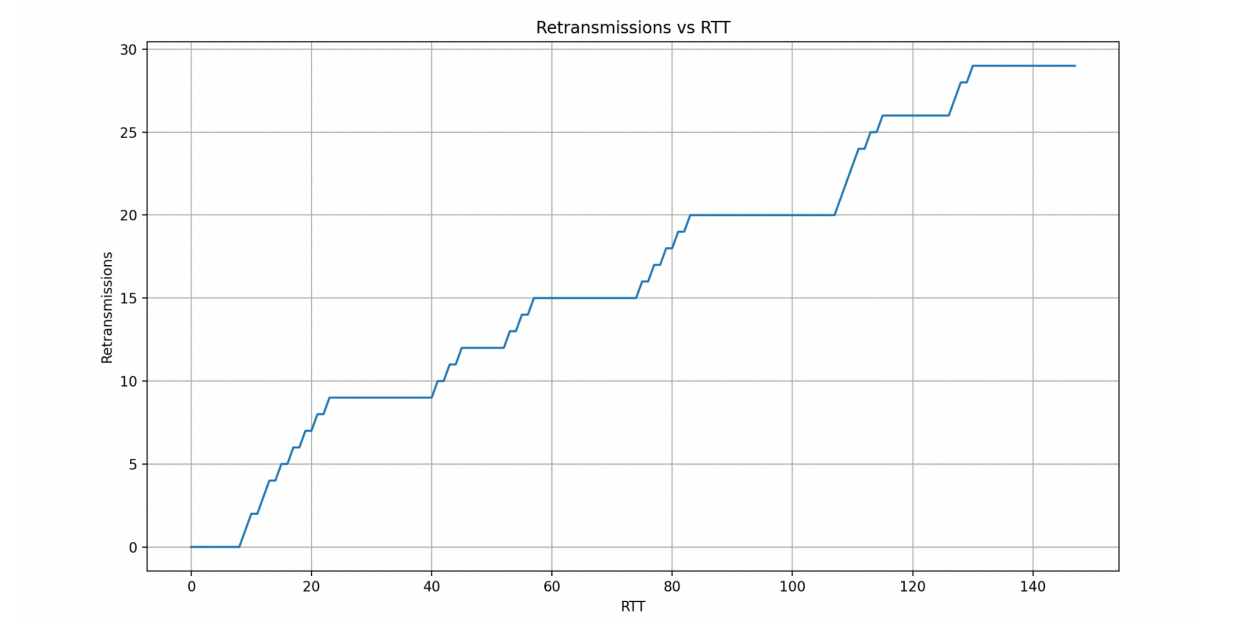


50%

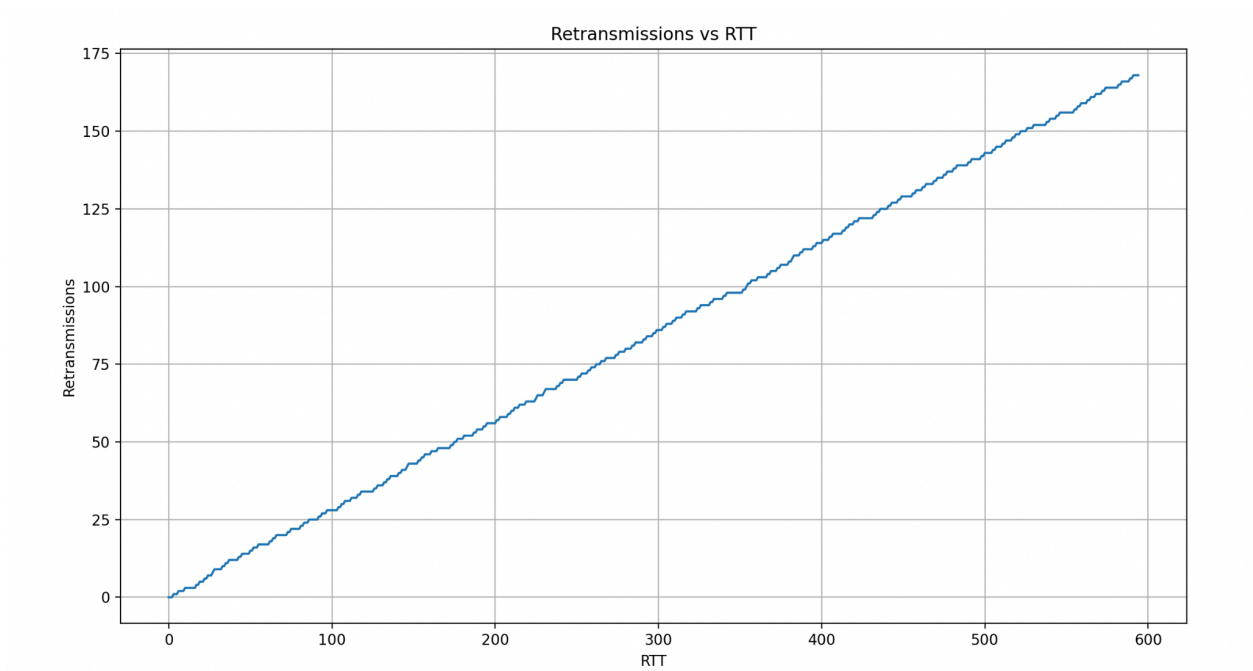


Retransmissions vs RTT

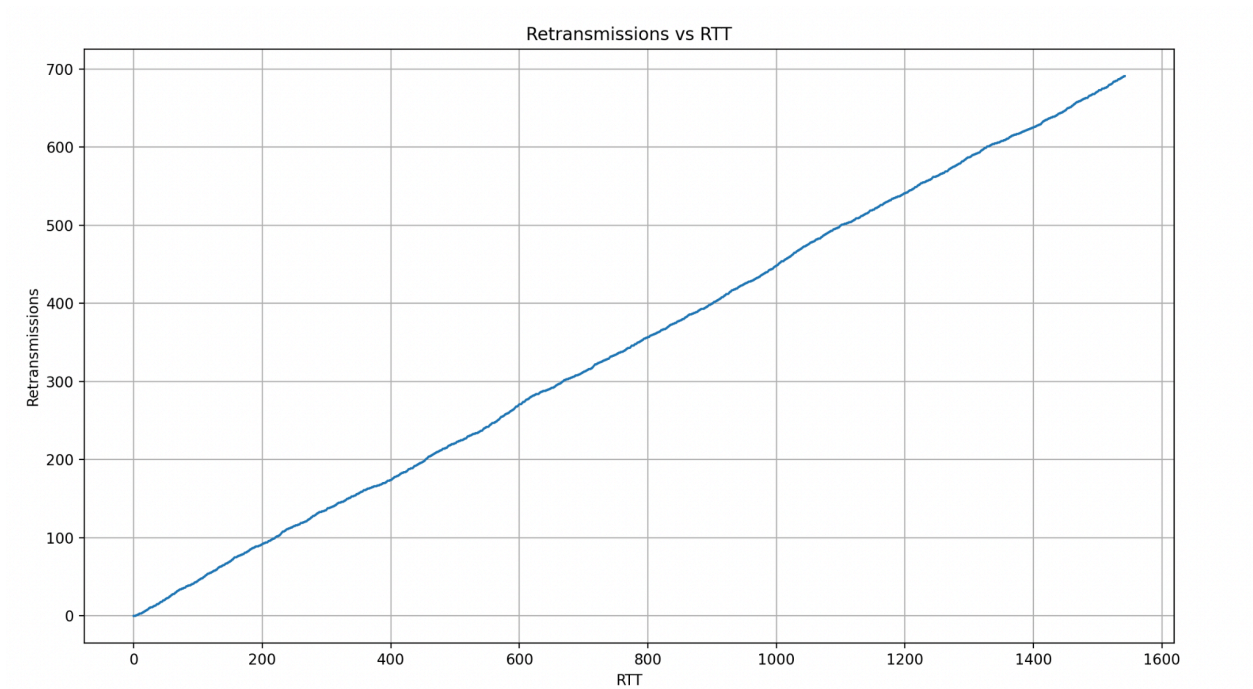
1%



10%:



50%:



Analysis of Graphs:

For the 1% and 10% graphs for cwnd, it can be seen that the first peak ended up being by far the highest. This is interesting, but it is clearly due to the behavior of ssthresh, which was originally set to an infinite value, so the congestion avoidance phase didn't occur until the first packet had to be retransmitted (which then brought ssthresh down to a real value). This likely didn't occur in the 50% packet drop graph because the first retransmit occurred too quickly before the cwnd could actually rise at all.

Generally in the cwnd graphs, as expected, the cwnd reached by far the highest for 1% drop rate, and by far the lowest for the 50% drop rate (of course, this is because when packets are dropped, the client has to retransmit them, lowering cwnd). For the same reasons, and for both cwnd and retransmits, the RTT reached the highest (ie, took more time) for 50% and was the quickest for 1%.

All the retransmit graphs follow a similar linear pattern; however, if you pay attention to each graph's scale, it can be seen that the proportion of retransmissions to RTT is the highest for 50%, and lowest for 1%. This is because dropped packets lead to more transmissions, and of course, 50% has the highest amount of dropped packets, and 1% the least.

Challenges and Lessons:

One challenge was that it was hard to tell how correct the system was working when testing. Reasonably early on, I was able to get the output file to match the input file, which by all means, I was happy with. However, it was clear that the behavior was incorrect. For example, in one of my test runs, I noticed that the server was signaling that basically every packet received was out of order (and thus, eventually causing the client to constantly retransmit packets). However, I didn't have anything concrete to tell what part of my code was wrong, so I had to work hard to manually inspect my code to check for issues, where I found that my client wasn't correctly updating send_base, an important variable that contained the first chunk number in the next batch of chunks to be sent. Ultimately this was a pretty good exercise at reviewing faulty code without obvious bugs.

Originally, I had overthought my timeout handler. In my original solution, I would keep track of a dictionary, storing every unreceived ACK and how long ago the packet number had been sent out. Then, the handler would loop through the dictionary to check each individual unreceived

ACK. However, this was better handled by purely checking the last sent ACK entirely, which is the expected behavior.